DSA REFERENCE

Backtracking Failure Modes

Common wrong turns, debugging cues, and complexity traps.

Build one candidate incrementally, abandon invalid prefixes early, and undo each choice before trying the next.

VERSION 1.0 REVIEW
REVIEWED THROUGH 2026-07-18
STATUS HUMAN REVIEW REQUIRED

DSA / BACKTRACKING

DSA

Backtracking Failure Modes

CORE IDEA

Build one candidate incrementally, abandon invalid prefixes early, and undo each choice before trying the next.

Recognition cues

+ enumerate combinations or permutations
+ constraint satisfaction
+ all valid arrangements
+ small exponential search

Implementation sequence

+ define choice set
+ prune before descending
+ choose
+ recurse
+ undo symmetrically

Invariant

The current path is a valid partial candidate and all mutable state reflects exactly that path.

Worked reasoning

For combinations summing to a target, sort candidates, skip duplicates at the same depth, and stop when the remainder is negative.

Complexity

Output-sensitive exponential time; stack and path space are bounded by decision depth.

Failure modes

- [ ] forgetting to undo
- [ ] copying excessive state
- [ ] duplicate outputs
- [ ] weak pruning
- [ ] confusing subsequences with permutations

Practice ladder

Backtracking Failure Modes: referenced problems

Open the official problem page, classify the pattern, write the invariant, and predict the complexity before coding.

#	Problem	Deliberate-practice prompt
1	**Subsets**	State the cue, invariant, edge case, and proof before implementation.
2	**Permutations**	State the cue, invariant, edge case, and proof before implementation.
3	**Combination Sum**	State the cue, invariant, edge case, and proof before implementation.
4	**Word Search**	State the cue, invariant, edge case, and proof before implementation.
5	**N-Queens**	State the cue, invariant, edge case, and proof before implementation.

Before coding

+ Pattern cue
+ State
+ Invariant
+ Boundary convention

Before finishing

+ Dry run
+ Edge case
+ Time proof
+ Space proof

PUBLISHING STATUS
REVIEW GATE: STABLE TECHNICAL. AI-produced draft. A named human must verify technical accuracy, sourcing, freshness, and editorial quality before publication. Reviewed-through date: 2026-07-18.